APPLIED GPT CAPABILITY PROGRAM FOR ENGINEERING PRACTITIONERS (GPT CODERS TRACK)

Capstone Projects

Guidelines, Shark Tank Review and Project Catalogue

Prepared for C5i, in partnership with Ambilio

01 ABOUT THE CAPSTONE PROJECTS

The capstone is what makes the ten days applied. The sessions teach how to build and ship production software on the OpenAI stack with Codex; the capstone is where a team takes a real problem from a C5i industry and practice, and builds a working system for it end to end — specified, built, reviewed, evaluated, secured, deployed and defended.

It is deliberately not an exercise. Every brief in this catalogue is shaped like work C5i actually delivers, and the standard of done is the standard the program teaches: a system that runs from a clean clone, carries measured results against a baseline, survives a security review and can be rolled back. Nothing here is disposable at the end of the ten days.

> Every team finishes the program holding a high-value proof of concept it built and can defend — a working system on a real C5i problem shape, engineered and evidenced well enough that C5i can take it further into a client-level solution if it chooses to.
>
> The three highest-scoring projects receive a certificate of appreciation, awarded alongside the program certification.

WHAT IT GIVES THE LEARNER

- Proof of capability, not attendance — a system you built on the OpenAI stack with Codex and defended in front of practitioners, rather than a course completed.
- The full path once, under supervision — problem to BRD to design to plan to build to review to evaluation to deployment, on one codebase, with the failures happening where they are cheap.
- Judgement about when the model or Codex is wrong — the review habits, the gates and the session discipline that separate an engineer using Codex from one being carried by it.
- A portfolio artefact that stands up to questioning, with the evidence pack behind it: review log, evaluation set, security triage, runbook.
- Individual recognition — scoring is per person, so what you personally understood and built is what is on record.

WHAT IT GIVES C5I

- Applied proof that the capability transferred — shipped, measured systems on the OpenAI stack rather than assessment scores alone.
- A portfolio of proofs of concept mapped to C5i's own industries and service practices, each with the evidence needed to judge whether it is worth taking further.
- Any project that earns it can be scaled into a client-level solution, because it was built to a production standard from the start rather than retrofitted afterwards.
- A per-individual read on preparedness and depth across the cohort — who is ready to lead OpenAI-based delivery on a client engagement and who needs more ground.
- Reusable engineering assets — AGENTS.md files, skills, MCP servers, evaluation sets and CI gates that outlive the program.

02 CAPSTONE GUIDELINES — COMMON TO EVERY PROJECT

Every team takes one brief from this catalogue, builds it with Codex and the OpenAI platform to the standard taught across the program, and defends it in a Shark Tank Review Presentation before a practitioner panel. The rules in this section apply to every brief; the catalogues in Sections 03 to 08 are industry-specific.

The brief tells you the business problem, what to build and the numbers you will be judged on. It does not tell you the architecture. Choosing the parts of the stack, specifying it, planning it, driving Codex to build it, reviewing what it produced, and proving the result is safe and operable is the work.

CAPSTONE AT A GLANCE

Catalogue	41 briefs across six industries and ten C5i service practices. Each brief is tagged to one of each.
Project source	One brief per team, chosen from this catalogue. A brief is taken by one team only, except Project 34, which several teams may take, each with a different use case.
Team size	Teams of three to four participants.
Industries	Banking & Financial Services · Consumer Goods · Retail · Technology · Healthcare & Life Sciences · Manufacturing.
Build window	From Day 3 alongside the sessions, then one week after Day 10 with no sessions running, as dedicated preparation time.
Data	Synthetic or public data only — no real customer data, no PII or PHI, and no C5i or client data of any kind. Generate realistic, real-format test data.
Environment	Codex as the build environment across the CLI, IDE, cloud and ChatGPT desktop app. The OpenAI API platform — Responses API, built-in tools, Agents SDK, MCP, Apps SDK and Realtime API — for the application itself.
Defence	Shark Tank Review Presentation of Capstone Projects — 20 minutes per team: 15 minutes presented by the team and 5 minutes of panel Q&A. Every team member presents.
Scoring	100-point weighted rubric across six criteria, scored individually for every member of the team.

TEAM AND EXECUTION

- Teams of three to four. Every member owns an area of the build and defends it at the panel.
- One brief per team. Pick the industry you can reason about — the panel will challenge the business case, not only the code.
- Use synthetic or public data only. Generate it in the real format, at a volume that makes the engineering decisions real.
- The build must carry the competencies taught across the ten days — not one day's technique demonstrated ten times.
- Codex does the typing. You own the result: what merges, what ships, and what it does in production is yours to defend.

WHAT “DONE” MEANS

- It runs — a working end-to-end system, demonstrated live from a clean clone, not slideware and not a notebook of fragments.
- It was engineered, not prompted — AGENTS.md, repository standards, skills, CI gates, the BRD, the design and the task plan all in version control, with a review record for what Codex produced.
- It is measured — real numbers against the brief's success metrics and the baseline they are compared to, plus an evaluation gate in CI that can block a merge.
- It is secure — scanning clear or explicitly risk-accepted, injection tested, project-scoped keys, secrets in a manager and out of prompts, traces and agent context, and Codex's sandbox and approval settings stated.

- It is operable — deployed to a non-production target with health checks, a demonstrated rollback and an operating runbook.
- It is honest — limitations, residual risk and what the system deliberately refuses to do, stated by the team before the panel has to ask.

PRE-PITCH DELIVERABLES

Submitted before the panel session. Anything not submitted cannot be claimed in the presentation.

- Repository that builds and runs from a clean clone, with a README covering setup, run instructions and an architecture diagram.
- Specification pack — BRD with testable acceptance criteria, solution design, decision records and the sequenced task plan.
- Agent engineering record — AGENTS.md, reusable skills and CI gates in version control, plus the review log for Codex-generated changes, Codex review and Codex Security findings, and the defects caught.
- Evaluation report with the metrics named in the brief, the baseline they are measured against, and the evaluation set itself.
- Security evidence — scan output and triage, injection test results, secrets and API-key handling, and the sandbox and approval boundary you set for Codex.
- Deployment evidence — non-production deployment, health checks, a demonstrated rollback, cost and usage telemetry, and an operating runbook.
- A 15-minute deck and a one-page summary of architecture, trade-offs and limitations.
- A working demo — live, with a recorded fallback in case of network failure.

02.1 SHARK TANK REVIEW PRESENTATION OF CAPSTONE PROJECTS — 20 MINUTES

Fifteen minutes presented by the team and five minutes of panel Q&A, one week after Day 10. Present the working system and walk its design, the parts of the OpenAI stack used and why, the way Codex was driven and controlled, the review and evaluation evidence, the security posture, cost and the deployment. The panel challenges the team live and the team defends its decisions. Every team member presents, and defends their own area under questioning. A system that does not run live cannot pass the demonstration criterion.

FIFTEEN-MINUTE PRESENTATION — SUGGESTED SPLIT

TIME	SEGMENT
2 min	Problem and business case — the pain, who carries it, and the cost of the status quo.
2 min	Solution design — the BRD, the parts of the stack you chose and rejected, and the task plan Codex executed.
2 min	How Codex was used and controlled — AGENTS.md and standards, cloud delegation, session discipline, and what you refused to merge.
5 min	Live demonstration — the working system on real-format synthetic inputs.
3 min	Evidence — code review, evaluation against the baseline, security posture, deployment and rollback.
1 min	Limitations and next steps — what it does not do yet, and what you would do next.
5 min	Panel Q&A — the panel tests architecture, stack choices, evaluation, cost and security. Every member answers on their own area.

EVALUATION RUBRIC

CRITERION	WEIGHT	WHAT IS ASSESSED
Functionality	25%	The system works end to end, runs live from a clean clone, and does what the brief asked.
Engineering and delivery discipline	20%	Standards and project setup for Codex; specification and planning; how Codex was driven and what was allowed to merge; security, deployment and a demonstrated rollback.
Evaluation and evidence	20%	Measured results against a stated baseline, a working evaluation gate in CI, and a review record showing defects were caught.
Relevance to C5i	10%	How closely the build maps to work C5i actually delivers, and whether the business case holds up under challenge.
Subject matter understanding	15%	Command of the OpenAI stack and the problem domain — why these components, what the alternatives were, and where the method breaks. Tested individually under questioning.
Presentation quality	10%	Clarity of the pitch and the deck, and the quality of answers under Q&A.

> Scoring is individual, not by team. The project is built and presented as a group, but every criterion is scored separately for each member of the group, on the area they own and on what they can defend under questioning.
>
> A strong system carried by one or two members will not earn the same score for the rest. Split the build, split the presentation, and make sure every member can answer for the whole design — not only the part they wrote.

PRESENTATION DECK

Five to six slides, no more. The deck supports the live demonstration; it does not replace it.

- Problem statement — the brief, who carries the pain, and the cost of the status quo.
- Solution — what you built and the design decisions behind it, including the ones you rejected.
- Methodology — how the work was specified, planned and driven with Codex, and how the output was reviewed.
- Experiment and outcomes — what you tried, what did not work, and what changed as a result.
- Results and evaluation — the metrics from the brief against the baseline, plus security and deployment evidence.
- Conclusion and future scope — limitations, residual risk, and what would take this from a proof of concept to production.

02.2 COVERAGE ACROSS C5I PRACTICES

Every brief is tagged to one C5i service practice and one industry. The catalogue is built so that a cohort working across it exercises the full practice portfolio rather than clustering on one.

SERVICE PRACTICE	PROJECT NUMBERS
AI & Data Strategy / Consulting	08, 27, 41
Generative AI & Agentic AI	02, 07, 09, 17, 26, 29, 34, 37
AI/ML Solutions & Applications	05, 11, 30, 36
Data Engineering & Data Modernization	03, 10, 18, 31, 38
Analytics & Decision Intelligence	01, 32, 40

SERVICE PRACTICE	PROJECT NUMBERS
Customer & Digital Analytics	06, 16, 25
Marketing & Sales Analytics	13, 20, 22
Supply Chain & Operations Analytics	14, 19, 23, 35
Pricing & Revenue Optimization	15, 24, 33
AI-powered Automation & Workflows	04, 12, 21, 28, 39

INDUSTRY	PROJECT NUMBERS
Banking & Financial Services	8 projects — 01, 02, 03, 04, 05, 06, 07, 08
Healthcare & Life Sciences	6 projects — 09, 10, 11, 12, 13, 14
Retail	7 projects — 15, 16, 17, 18, 19, 20, 21
Consumer Goods	6 projects — 22, 23, 24, 25, 26, 27
Technology	7 projects — 28, 29, 30, 31, 32, 33, 34
Manufacturing	7 projects — 35, 36, 37, 38, 39, 40, 41

03 BANKING & FINANCIAL SERVICES

8 capstone briefs. Briefs for regulated decisioning, servicing and reporting systems, where an unexplained decision is a compliance problem and not only a quality one. Each is scoped for a team of three to four from Day 3 to the panel, on synthetic data, and states the business problem, what to build, and the success metrics it will be judged on.

01 BANKING & FINANCIAL SERVICES · ANALYTICS & DECISION INTELLIGENCE **Loan Covenant Monitoring Console**	
Business problem	Commercial loan covenants sit in agreements that run to a hundred pages, and compliance is checked quarterly by an analyst re-reading the terms against the borrower's financials. A breach is found when the next review happens, not when the numbers move, and two analysts reading the same agreement routinely extract different thresholds.
What to build	Write the BRD with testable acceptance criteria into the repository before any code, with AGENTS.md carrying the standards Codex reads on every run. Extract covenant terms from synthetic loan agreements on the Responses API with Structured Outputs under a strict schema, and validate every threshold against the clause it came from. Compute covenant headroom from synthetic quarterly financials in application code, not in the model, and surface breaches and near-breaches in an operations console with the clause and calculation behind each. Put an extraction evaluation set in CI as a merge gate and deploy with a canary and a demonstrated rollback.
Success metrics	Covenant-term extraction accuracy against a labelled agreement set, breach-detection recall, clause-citation correctness, headroom calculation exactness, blocked-merge demonstration.

Key techniques	• Structured Outputs with strict schemas over long legal documents • Clause-level citation for every extracted term • Deterministic calculation kept in code, not in the model • Extraction evaluation set as a CI merge gate • Canary release with automated rollback

02 BANKING & FINANCIAL SERVICES · GENERATIVE AI & AGENTIC AI

Card Services Voice Agent

Business problem	Card servicing calls — lost cards, declined transactions, travel notices — are short, repetitive and handled by people working a queue. Customers wait longest for the simplest requests, and a voice system that mishears a card action or acts without confirmation is a fraud and complaint problem, not a convenience one.
What to build	Build a voice agent on the Realtime API that handles a defined set of card servicing intents against mock card systems exposed as typed tools. Tune turn detection for callers who pause mid-sentence, and hold a stated latency budget from end of speech to first response. Require spoken confirmation, read back and enforced in application code, before any card action executes, and hand off to a human with a structured summary when an intent is out of scope or identity checks fail. Log every tool call against the call session, and evaluate on a scripted call set that includes noisy audio, interruptions and ambiguous requests.
Success metrics	Intent-resolution rate on the scripted call set, wrong-action rate with a target of zero, end-of-speech to first-response latency, handoff correctness, confirmation-enforcement test pass.
Key techniques	• Realtime API voice agent with mid-conversation tool calls • Turn detection tuning and latency budgets • Confirmation enforced in code before any write action • Structured handoff to a human agent • Scripted call-set evaluation including noisy audio

03 BANKING & FINANCIAL SERVICES · DATA ENGINEERING & DATA MODERNIZATION

Statement Generation Batch Modernisation with Codex

Business problem	Customer statements are produced by a nightly batch of scripts that has grown for a decade and that nobody on the current team wrote. Fee and interest presentation rules are buried in code, every regulatory wording change is patched in place, and nobody can say what a change will break until a customer complains about their statement.
What to build	Take the supplied legacy statement batch repository you did not write. Use Codex to produce an architecture map, a dependency and impact analysis, and the business rules reconstructed from code alone — then validate every rule against the supplied rule sheet, because the agent will state some of them confidently and wrongly. Write characterisation tests that pin current statement output, then have Codex re-implement one statement path as a modern, tested service held inside a defined blast radius. Prove output equivalence against the legacy path on a synthetic account set before anything is switched, with Codex review and a human sign-off on the migration pull requests.
Success metrics	Rule-reconstruction accuracy against the supplied key, characterisation-test coverage of the migrated path, output equivalence rate against legacy, impact-analysis completeness, review defects caught before merge.

Key techniques	• Codex-driven comprehension of an unfamiliar repository • Business-rule reconstruction validated against a known key • Characterisation testing before any refactor • Blast-radius control on agent-driven change • Output-equivalence verification against the legacy path

04 BANKING & FINANCIAL SERVICES · AI-POWERED AUTOMATION & WORKFLOWS

KYC Document Intake Workflow

Business problem	Customer onboarding stalls on identity and address documents that arrive as phone photos, scans and PDFs. Operations staff key fields by hand, check them against the application and chase customers for re-submission. A field misread is a compliance gap; a valid document rejected is a lost customer.
What to build	Build a workflow service that ingests synthetic identity and address documents as images and PDFs, extracts the required fields with a vision-capable model under a strict schema, and validates them against the application record and format rules — dates, checksums, address consistency. Declare an unreadable or partial document as a gap rather than passing it. Route low-confidence or mismatched cases to a human review queue with the image region and reason attached. Keep personal data out of logs and traces, and put the extraction and validation sets in CI as a merge gate.
Success metrics	Field-extraction accuracy on a labelled document set, false-acceptance rate, gap-declaration correctness, straight-through rate at the chosen thresholds, personal data absent from traces.
Key techniques	• Vision input with schema-constrained extraction • Rule-based validation against the application record • Confidence thresholds routing to human review • Personal data kept out of logs and traces • CI merge gates on extraction quality

05 BANKING & FINANCIAL SERVICES · AI/ML SOLUTIONS & APPLICATIONS

Transaction Enrichment Service at Batch Scale

Business problem	Card and account transactions arrive as cryptic merchant strings that customers cannot recognise and analytics cannot group. The bank enriches them with a rules table that misses new merchants daily, and the obvious fix — sending every transaction to a large model — has never been costed against the volume.
What to build	Build an enrichment service that classifies synthetic transaction descriptions into merchant, category and channel under a strict schema, running the overnight backlog through the Batch API and the low-volume real-time path through the Responses API with prompt caching. Compare at least two models and reasoning settings on a labelled set, and choose on accuracy and cost per thousand transactions, not on accuracy alone. Cache confirmed merchants so the model is called only for unseen strings. Report cost, latency and accuracy per configuration, and gate merges on a labelled-set regression.
Success metrics	Category accuracy against the rules-table baseline, cost per thousand transactions per configuration, cache hit rate, real-time p95 latency, batch completion time, blocked-merge demonstration.

Key techniques	• Batch API for high-volume offline workloads • Prompt caching on the real-time path • Model and reasoning-setting comparison on cost and accuracy • Result caching to avoid repeat model calls • Labelled-set regression gate in CI

06 BANKING & FINANCIAL SERVICES · CUSTOMER & DIGITAL ANALYTICS

Complaints Insight App Inside ChatGPT

Business problem	Complaints analysts answer leadership questions — what is driving complaints this month, which products, which channels — by exporting data, building pivots and writing a summary. The questions repeat, the answers arrive days late, and two analysts produce different numbers from the same export.
What to build	Build a ChatGPT app with the Apps SDK, backed by an MCP server that exposes a synthetic complaints dataset through read-only typed tools: counts by cause, trends, and sample complaints behind each figure. Compute every number in the server, never in the model, so two people asking the same question get the same answer. Render results with UI components that show the figure, the definition used and the sample records behind it. Scope access per user, log every tool call, and evaluate on a question set with verified answers, including questions the data cannot answer.
Success metrics	Answer exactness against verified figures, consistency of repeated answers, refusal correctness on unanswerable questions, tool-call audit completeness, p95 response time.
Key techniques	• Apps SDK app running inside ChatGPT • MCP server with read-only typed tools • Figures computed server-side, never by the model • Per-user access scoping and audit logging • Verified question-set evaluation

07 BANKING & FINANCIAL SERVICES · GENERATIVE AI & AGENTIC AI

Card Dispute Handling Agent

Business problem	Card disputes arrive by chat, email and phone and must be sorted into fraud, merchant dispute or billing error, each with different evidence, rules and deadlines. Misclassification sends a case down the wrong path and costs days; a chargeback raised on a weak case costs money and merchant goodwill.
What to build	Build a triage agent with the OpenAI Agents SDK that classifies synthetic dispute cases and hands off to specialist agents for fraud, merchant dispute and billing error, each with its own tools against mock card and case systems. Add input guardrails that stop out-of-scope or abusive requests, and an approval step before any chargeback is raised. Set run control — iteration limit, budget ceiling and stop-and-escalate — and keep tracing on for every run. Compare behaviour and cost against a LangGraph version of the same flow on one case type.
Success metrics	Classification accuracy on a labelled case set, handoff correctness, chargeback-approval enforcement, cost per resolved case, budget-ceiling adherence.

Key techniques	• Agents SDK triage with specialist handoffs • Input guardrails and approval before risky actions • Run control with iteration and budget limits • End-to-end tracing of agent runs • Orchestration comparison against LangGraph

08 BANKING & FINANCIAL SERVICES · AI & DATA STRATEGY / CONSULTING

Model Routing Business Case for a Contact Centre

Business problem	A contact centre wants AI across summarisation, classification, drafting and knowledge answers, and the default plan is one large model for everything. Nobody has measured which tasks need it, what each task costs at real volume, or what quality is actually lost by routing the easy ones elsewhere.
What to build	Take the contact centre's task portfolio through a structured analysis: volumes, quality bars and risk per task, and candidate model and reasoning settings for each. Build an evaluation set per task and measure quality, latency and cost for each candidate, including batch and caching options where they apply. State plainly which tasks should use a smaller model, which need the larger one and which should not use a model at all. Build the cost and quality case with sensitivity on the assumptions that move the answer, then build a working router for the two highest-volume tasks, to the same engineering, security and deployment standard as every other project.
Success metrics	Evaluation coverage per task, cost per thousand interactions against a single-model baseline, quality loss per routed task, defensibility of the routing decision under panel challenge, working router delivered.
Key techniques	• Task-level evaluation sets across a portfolio • Model and reasoning-setting selection on measured trade-offs • Cost model with sensitivity analysis • Do-not-use-a-model conclusions where justified • Working router for the selected tasks

03 HEALTHCARE & LIFE SCIENCES

6 capstone briefs. Briefs where a confidently wrong answer is a safety or compliance event, and what the system refuses is judged as closely as what it produces. Each is scoped for a team of three to four from Day 3 to the panel, on synthetic data, and states the business problem, what to build, and the success metrics it will be judged on.

09 HEALTHCARE & LIFE SCIENCES · GENERATIVE AI & AGENTIC AI

Clinical Trial Protocol Assistant

Business problem	Site staff answer eligibility and procedure questions against trial protocols that are amended several times during a study. The answer to 'can this patient be enrolled' depends on which amendment is in force at that site, and an answer drawn from a superseded version is a protocol deviation.

What to build	Build retrieval over a synthetic set of protocols and amendments using file search over vector stores, with metadata filters for study, site and amendment effective date applied before retrieval. Cite the specific section and amendment in every answer, and refuse when the protocol does not cover the question rather than answering from adjacent text. Curate a golden set with superseded-amendment traps, out-of-scope questions and near-duplicates across studies. Then build the same feature on your own pgvector hybrid retrieval and decide, with numbers, which one ships.
Success metrics	Answer accuracy on the golden set, amendment-version correctness, refusal correctness, citation-grounding rate, cost and latency per query for each retrieval option.
Key techniques	• File search over vector stores with metadata filters • Version-aware citation to section and amendment • Calibrated refusal on uncovered questions • Golden set with superseded-version traps • Built-in versus own retrieval decision on measured results

10 HEALTHCARE & LIFE SCIENCES · DATA ENGINEERING & DATA MODERNIZATION

Laboratory Report Digitisation Pipeline

Business problem	Laboratory results from partner sites arrive as scanned reports and PDFs in a dozen layouts. They are keyed into the data platform by hand, units vary by lab, and a transcription error in a value is found only when an analysis looks wrong — if it is found at all.
What to build	Build an ingestion pipeline that extracts synthetic laboratory report values with a vision-capable model under a strict schema, normalises units, and validates every value against reference ranges, plausibility rules and cross-field consistency before it lands. A failing record does not land; it goes to a review queue with the image region attached. Process the historical backlog through the Batch API and new reports through the real-time path. Track lineage from every stored value to its source page, and inject low-quality scans to measure which checks catch the errors.
Success metrics	Value-extraction accuracy on a labelled report set, unit-normalisation correctness, detection rate on injected bad scans, lineage completeness, backlog cost and throughput.
Key techniques	• Vision-based extraction from scanned documents • Range, plausibility and consistency checks as landing gates • Batch API for backlog processing • Lineage from stored value to source page • Fault injection with degraded scans

11 HEALTHCARE & LIFE SCIENCES · AI/ML SOLUTIONS & APPLICATIONS

Prior Authorisation Pre-Check Service

Business problem	Prior authorisation requests are rejected for missing documentation or unmet criteria that could have been caught before submission. Each rejection delays treatment and adds rework for clinic staff, and the payer criteria change often enough that nobody keeps a current checklist.

What to build	Build a service on the Responses API that takes a synthetic authorisation request, retrieves the applicable payer criteria, and returns a structured pre-check: each criterion, whether the submitted evidence meets it, and what is missing. Call a deterministic rules service through function calling for anything that can be computed, so the model is used only for reading evidence. Validate the structured output against business rules before it is returned. Serve it behind a versioned API with an evaluation gate in CI and a tested rollback.
Success metrics	Criterion-level accuracy against a labelled request set, missing-evidence recall, false-pass rate, p95 latency, rollback success.
Key techniques	• Responses API with Structured Outputs • Function calling to a deterministic rules service • Business-rule validation after schema validation • Versioned API with CI evaluation gate • Tested rollback path

12 HEALTHCARE & LIFE SCIENCES · AI-POWERED AUTOMATION & WORKFLOWS

Patient Appointment Voice Line

Business problem	Clinic phone lines are dominated by rescheduling and appointment questions, and patients with genuinely urgent symptoms wait in the same queue. An automated line that books well but misses a patient describing a red-flag symptom is a safety failure, regardless of how many calls it handled.
What to build	Build a voice line on the Realtime API that handles appointment booking, rescheduling and preparation questions against a mock scheduling system through typed tools. Detect red-flag symptom language and escalate immediately to a nurse line with a structured summary, without attempting clinical advice. Confirm every booking change by reading it back before it is written. Evaluate on a scripted call set that includes red-flag mentions buried mid-call, accents, interruptions and patients who change their mind.
Success metrics	Red-flag escalation recall with a target of zero misses, booking-change accuracy, end-of-speech to response latency, containment rate on routine calls, confirmation-enforcement test pass.
Key techniques	• Realtime API voice workflow with tool calls • Safety escalation on red-flag language • Read-back confirmation before writes • Structured handoff summaries • Scripted call-set evaluation with safety cases

13 HEALTHCARE & LIFE SCIENCES · MARKETING & SALES ANALYTICS

Field Representative Pre-Call App in ChatGPT

Business problem	Field representatives prepare for visits by piecing together engagement history, content use and territory data from several tools. Preparation quality varies widely, and suggesting off-label topics or unapproved material is a compliance breach, not a sales tactic.

What to build	Build a ChatGPT app with the Apps SDK that gives representatives a pre-call view for a contact: engagement history, approved content to use, and suggested discussion points, served from synthetic data through an MCP server. Restrict content suggestions to an approved-material list held as configuration, and add an output check that blocks off-label or unapproved references before anything is shown. Capture which suggestions a representative used or rejected, and feed rejections into the evaluation set. Evaluate on contacts with known outcomes and on planted compliance traps.
Success metrics	Suggestion acceptance against a static-list baseline, compliance-trap block rate with a target of zero leaks, approved-material coverage, feedback capture rate, p95 response time.
Key techniques	• Apps SDK app with an MCP server behind it • Approved content held as configuration • Output checks blocking non-compliant suggestions • Feedback capture into the evaluation set • Planted compliance-trap evaluation

14 HEALTHCARE & LIFE SCIENCES · SUPPLY CHAIN & OPERATIONS ANALYTICS

Clinical Supply Shortage Early Warning Agent

Business problem	Shortages of trial drug and comparator supply are spotted when a depot runs low, by which time resupply is already late. Signals exist earlier — enrolment running ahead of plan, supplier notices, shipping delays — but they sit in different places and nobody joins them.
What to build	Build an agent that combines synthetic depot inventory and enrolment data, exposed through MCP tools, with web search over supplier and logistics notices, and produces a ranked shortage risk list per depot with the evidence behind each. Compute stock-out dates in code from consumption rates, not in the model. Cite every external claim to its source and mark unverifiable ones. Set run limits and a budget ceiling, and evaluate on scenarios with planted shortages to measure detection lead time against a stock-level alert baseline.
Success metrics	Planted-shortage detection rate and lead time against a stock-level baseline, stock-out date accuracy, external citation correctness, cost per run, budget-ceiling adherence.
Key techniques	• Agent combining internal tools and web search • Deterministic forecasting kept in code • Citation and verifiability marking for external claims • Run control with budget limits • Planted-scenario evaluation of lead time

03 RETAIL

7 capstone briefs. Briefs for high-volume decisioning and store-facing systems, where latency, consistency and a defensible method matter more than model sophistication. Each is scoped for a team of three to four from Day 3 to the panel, on synthetic data, and states the business problem, what to build, and the success metrics it will be judged on.

15 RETAIL · PRICING & REVENUE OPTIMIZATION

Competitor Price Intelligence Service

Business problem	Pricing teams track competitor prices from screenshots and manual checks of a few hundred products, matched to their own catalogue by eye. Coverage is thin, matches are wrong often enough that nobody trusts the price index, and a pricing decision taken on a mismatched product is worse than no data.
What to build	Build a service that extracts price, pack size, promotion and availability from synthetic competitor page snapshots and screenshots under a strict schema, using vision input where the page is an image. Match each item to the own catalogue with a confidence score, send the uncertain band to a review queue, and compute a price index only from confirmed matches. Run the weekly refresh through the Batch API. Report unmatched and uncertain volume as first-class numbers, and gate merges on extraction and matching accuracy against a labelled set.
Success metrics	Extraction accuracy, match precision and recall against a labelled pair set, review-queue volume at the chosen threshold, price-index reproducibility, cost per refresh.
Key techniques	· Vision and HTML extraction under strict schemas · Confidence-scored matching with a human review band · Index computed only from confirmed matches · Batch API for scheduled refresh · Labelled-set accuracy gates in CI

16 RETAIL · CUSTOMER & DIGITAL ANALYTICS

Review and Return Reason Analyser

Business problem	Product reviews and return reasons carry the earliest signal of a quality or sizing problem, and they are read as a monthly sentiment score. A defect affecting one supplier's batch disappears into a category average until return rates have already risen.
What to build	Build a service that classifies synthetic reviews and return reasons into a product-defect taxonomy under a strict schema, processes history through the Batch API and new feedback in near real time, and links each classification to the product, supplier and batch. Detect emerging clusters per product and supplier with thresholds computed in code. Build a dashboard that opens any cluster to the reviews behind it. Evaluate on data with planted defect clusters, and measure detection lead time against a sentiment-score baseline.
Success metrics	Taxonomy classification accuracy on a labelled set, planted-cluster detection rate and lead time, link-to-source accuracy, cost per thousand reviews, dashboard p95 load time.
Key techniques	· Schema-constrained classification at batch scale · Taxonomy design with a labelled evaluation set · Cluster detection computed in code · Drill-down from cluster to source records · Planted-signal evaluation of lead time

17 RETAIL · GENERATIVE AI & AGENTIC AI

Shopping Assistant as a ChatGPT App

Business problem	Shoppers increasingly start product discovery inside ChatGPT rather than on the retailer's site. A retailer that is not present there loses the visit; one that is present with wrong stock, wrong prices or an unconfirmed basket change loses trust faster than it gains sales.

What to build	Build a ChatGPT app with the Apps SDK, backed by an MCP server over a synthetic catalogue, inventory and basket service. Expose read tools for search, availability and price, and one write action that adds to a basket, confirmed explicitly by the user and enforced on the server. Render product results with UI components that show live stock and price from the tools, never from the model. Test crafted prompts that try to trigger the write action or alter prices, and log every action against the user session.
Success metrics	Search relevance on a labelled query set, price and stock accuracy against the source system, unconfirmed-write rate with a target of zero, injection-test pass rate, p95 tool latency.
Key techniques	· Apps SDK app with read and confirmed write actions · Server-side enforcement of user confirmation · Live data rendered from tools, not the model · Injection testing against write actions · Session-level action logging

18 RETAIL · DATA ENGINEERING & DATA MODERNIZATION

Product Attribute Enrichment Pipeline

Business problem	New products arrive from suppliers with incomplete and inconsistent attributes — colour, material, size range, care instructions. Filters on the site do not work for these products, and the merchandising team fills gaps by hand weeks after launch.
What to build	Build an enrichment pipeline that fills missing attributes for synthetic products from supplier descriptions and product images under a strict schema, with a controlled vocabulary per attribute. Assign a confidence to each filled value, write high-confidence values automatically and queue the rest for review with the evidence attached. Run incrementally with high-water-mark tracking, lineage from every filled value to its source, and the backlog through the Batch API. Inject contradictory descriptions and images and report how the pipeline handles them.
Success metrics	Attribute-fill accuracy on a labelled set, vocabulary conformance, review-queue volume at the chosen threshold, lineage completeness, cost per thousand products.
Key techniques	· Text and image inputs with controlled-vocabulary schemas · Confidence thresholds with a review queue · Incremental processing with high-water-mark tracking · Lineage from filled value to source · Contradiction injection testing

19 RETAIL · SUPPLY CHAIN & OPERATIONS ANALYTICS

Store Replenishment Exception Agent

Business problem	Automated replenishment produces a daily exception list — suppressed orders, forecast overrides, stock-out risks — that grows faster than planners can work it. Planners triage by store familiarity, and an exception ignored for two days becomes an empty shelf.

What to build	Build an agent with the Agents SDK that works synthetic replenishment exceptions through MCP tools over mock inventory, forecast and order systems: classify the cause, estimate the cost of ignoring it, and propose an action. Order changes are proposals that a planner approves; the agent cannot write them on its own. Add guardrails that stop the agent acting outside its store scope, run limits per exception, and tracing on every run. Present the queue ordered by cost of inaction, and evaluate classification and ordering against a labelled set with realised outcomes.
Success metrics	Cause-classification accuracy, ordering quality against realised impact, approval-enforcement test pass, cost per exception worked, p95 run time.
Key techniques	• Agents SDK agent working MCP tools • Approval before any order change • Scope guardrails per store • Run limits and tracing • Impact-ordered queue evaluated on realised outcomes

20 RETAIL · MARKETING & SALES ANALYTICS

Campaign Copy Service with Brand and Claims Guardrails

Business problem	Marketing teams generate campaign copy at volume with AI, and review has become the bottleneck. Copy that breaks brand rules is an annoyance; copy that makes an unsupported price or product claim is a regulatory and legal problem, and it is the kind reviewers miss under volume.
What to build	Build a copy-generation service on the Responses API that produces channel-specific variants from a synthetic campaign brief, with brand rules and approved claims held as version-controlled configuration. Add output guardrails that check every claim against the approved list and every price against the source system, and block rather than rewrite on failure. Score copy quality with an LLM-as-judge calibrated against human labels, and report the agreement rate. Gate merges on claim-check recall against a planted-violation set.
Success metrics	Planted-violation block rate, false-block rate, judge agreement with human labels, review time per approved variant against a manual baseline, blocked-merge demonstration.
Key techniques	• Generation with rules held as configuration • Output guardrails on claims and prices • LLM-as-judge calibrated against human labels • Planted-violation evaluation set • CI gate on guardrail recall

21 RETAIL · AI-POWERED AUTOMATION & WORKFLOWS

Store Associate Voice Assistant

Business problem	Store associates answer stock, location and order questions while serving customers, using a handheld device that needs both hands and several screens. The answer exists in the systems; reaching it takes longer than the customer is willing to wait.

What to build	Build a hands-free voice assistant on the Realtime API that answers stock, aisle location and order status questions through typed tools over mock store systems. Hold a latency budget suited to a customer standing in front of the associate, and keep answers short. Handle background noise and interruptions, and confirm before any order action such as a hold or transfer. Evaluate on a scripted set recorded with store-level noise, and report accuracy and latency by noise condition.
Success metrics	Answer accuracy by noise condition, end-of-speech to first-response latency, order-action confirmation enforcement, task completion time against a handheld baseline.
Key techniques	· Realtime API voice interface with tool calls · Latency budgeting for face-to-face use · Noise and interruption handling · Confirmation before order actions · Condition-split evaluation reporting

03 CONSUMER GOODS

6 capstone briefs. Briefs across trade, demand and revenue decisions, where the hard part is reconciling sources that never agree and being honest about what did not reconcile. Each is scoped for a team of three to four from Day 3 to the panel, on synthetic data, and states the business problem, what to build, and the success metrics it will be judged on.

22 CONSUMER GOODS · MARKETING & SALES ANALYTICS

Trade Promotion Post-Event Analyst

Business problem	Promotion post-mortems are argued rather than measured. Uplift is calculated in personal spreadsheets against baselines nobody agrees on, and the same promotion is reported as a success or a failure depending on who did the analysis.
What to build	Build an analyst service that answers post-event questions about synthetic promotions by running a fixed, documented uplift method in code — baseline definition, control selection, confidence interval — with the model choosing which analysis to run and explaining the result, not inventing the method. Use code interpreter or your own execution sandbox, and return the code and data behind every number. Refuse when the data for a period is incomplete. Validate against data with a planted effect and gate merges on recovering it within a stated tolerance.
Success metrics	Planted-effect recovery within tolerance, reproducibility across runs, refusal correctness on incomplete periods, method-change gate demonstration, analysis turnaround against a manual baseline.
Key techniques	· Fixed analysis method in code, explained by the model · Code execution with returned code and data · Refusal on incomplete data · Synthetic ground-truth validation · Method-change gates in CI

23 CONSUMER GOODS · SUPPLY CHAIN & OPERATIONS ANALYTICS

Supplier Confirmation and Delay Parser

Business problem	Supplier order confirmations, partial shipments and delay notices arrive as free-text emails and attachments. Planners read them and update the ERP by hand, so a delay notice read late becomes a production shortfall, and a duplicate update corrupts the order book.
What to build	Build a workflow that extracts confirmation, quantity, date and delay details from synthetic supplier emails and attachments under a strict schema, matches them to open purchase orders, and proposes updates through an MCP tool against a mock ERP. Make every write idempotent so a retried message cannot create a duplicate update. Route unmatched or conflicting messages to a planner with the evidence attached. Inject duplicates, partial failures and conflicting notices, and report which guarantees held.
Success metrics	Extraction accuracy on a labelled message set, PO-match precision, duplicate-update rate under forced retries, conflict-detection recall, planner time per message against baseline.
Key techniques	• Schema-constrained extraction from email and attachments • MCP tool integration with an ERP • Idempotent writes with retry testing • Conflict and mismatch routing to a human • Injected-failure testing

24 CONSUMER GOODS · PRICING & REVENUE OPTIMIZATION

Price Pack Architecture Copilot

Business problem	Price and pack decisions are debated with static spreadsheets that cannot answer the actual question — what happens to volume and margin if we move this price and hold that pack. Each scenario takes days, so few are tested, and the analysis arrives after the decision.
What to build	Build a copilot on the Responses API that turns a planner's question into calls to a deterministic elasticity simulator through function calling, and explains the result with the assumptions shown. The model never computes outcomes itself. Refuse scenarios outside the range the elasticity data supports, rather than projecting through empty space. Build a comparison-first interface that shows the scenarios that lose as well as the one that wins, and hold a performance budget so a scenario returns fast enough to use in a meeting.
Success metrics	Scenario computation time, out-of-range refusal correctness, tool-call argument accuracy on a labelled question set, reproducibility of scenario results, assumption-display coverage.
Key techniques	• Function calling to a deterministic simulator • Model limited to interpretation and explanation • Extrapolation bounds with explicit refusal • Comparison-first interface design • Performance budgeting for interactive use

25 CONSUMER GOODS · CUSTOMER & DIGITAL ANALYTICS

Voice-of-Consumer Retrieval Service

Business problem	Consumer reviews, ratings and care contacts hold specific answers — which pack format leaks, which flavour gets complaints in which region — but the team only sees monthly theme summaries. Specific questions go unanswered because nobody can search the detail.

What to build	Build retrieval over synthetic consumer feedback on PostgreSQL with pgvector, combining keyword and semantic search with a reranking stage and filters for product, pack, region and date applied before retrieval. Attribute every answer to the feedback records behind it. Build a labelled query set, measure each stage's contribution by ablation, and compare the result with the built-in file search tool on the same set. Expose it behind an API with caching, and report cost and latency per configuration.
Success metrics	Precision at k on the labelled set, per-stage ablation deltas, comparison against built-in file search, cache hit rate, p95 latency and cost per query.
Key techniques	• Hybrid retrieval with reranking on pgvector • Pre-retrieval metadata filters • Ablation testing of each pipeline stage • Comparison against built-in file search • Record-level attribution of answers

26 CONSUMER GOODS · GENERATIVE AI & AGENTIC AI

Pack Artwork Claims Checker

Business problem	Every pack artwork change is checked by hand against regulatory and brand claim rules for each market — nutrition claims, allergen statements, mandatory wording. Reviews are slow, and a non-compliant claim found after printing means a recall of packaging, not a correction.
What to build	Build a checker that reads synthetic pack artwork images with a vision-capable model, extracts every claim and mandatory statement under a strict schema, and checks each against the market's claim rules retrieved from a version-controlled rule library. Cite the rule behind every finding, and declare text it cannot read as a gap rather than passing it. Produce a review report a regulatory specialist signs off. Evaluate on artwork with planted violations and near-miss compliant wording.
Success metrics	Planted-violation recall, false-finding rate on compliant near-misses, rule-citation accuracy, gap-declaration correctness, review time against a manual baseline.
Key techniques	• Vision extraction from artwork images • Rule retrieval from a version-controlled library • Rule-cited findings with gap declaration • Specialist sign-off before release • Planted-violation and near-miss evaluation

27 CONSUMER GOODS · AI & DATA STRATEGY / CONSULTING

Generative AI Use-Case Portfolio Assessment

Business problem	Business functions bring a long list of generative AI ideas, and the ones that get built are the ones with the loudest sponsor. Few are assessed on volume, risk, data readiness or cost to run, so some expensive builds deliver little and some valuable ones never start.
What to build	Take a consumer goods company's candidate use cases through a structured assessment: value, feasibility, data availability, risk and cost to run at real volume. Build a small evaluation set for the top three and measure what the current OpenAI stack actually achieves on each, with cost per task. State plainly which should be built, which should wait and which should not use generative AI at all. Build the business case with bounds and sensitivity, then deliver a working, evaluated proof for the top candidate to the same standard as every other project.

Success metrics	Assessment completeness across the portfolio, measured quality and cost per task for the top three, defensibility of the ranking under panel challenge, working proof delivered.
Key techniques	• Portfolio assessment against defined criteria • Measured feasibility on small evaluation sets • Cost-to-run modelling at real volume • Business case with sensitivity analysis • Working proof for the selected use case

03 TECHNOLOGY

7 capstone briefs. Briefs for product, support and platform systems, including one that starts from a codebase nobody on the team wrote. Each is scoped for a team of three to four from Day 3 to the panel, on synthetic data, and states the business problem, what to build, and the success metrics it will be judged on.

28 TECHNOLOGY · AI-POWERED AUTOMATION & WORKFLOWS

Incident Triage Agent for Platform Operations

Business problem	On-call engineers spend the first half hour of every incident gathering the same evidence — recent deploys, error rates, logs, the relevant runbook — before any diagnosis starts. Under pressure the gathering is incomplete, and the runbook step that would have fixed it is the one nobody looked up.
What to build	Build an agent with the Agents SDK that assembles an incident brief from synthetic deploy history, metrics and logs exposed through read-only MCP tools, retrieves the matching runbook sections with citations, and proposes a ranked list of likely causes with the evidence behind each. The agent recommends; a human executes any remediation. Set run limits and a budget ceiling, trace every run, and guard against instructions planted in log content. Evaluate on a set of replayed incidents with known causes, and measure time-to-brief against a manual baseline.
Success metrics	Cause-ranking accuracy on replayed incidents, runbook citation correctness, time-to-brief against a manual baseline, injection-test pass, cost per incident.
Key techniques	• Agents SDK agent over read-only MCP tools • Runbook retrieval with citation • Human execution of any remediation • Injection defence on log and tool content • Replayed-incident evaluation

29 TECHNOLOGY · GENERATIVE AI & AGENTIC AI

Internal Engineering Knowledge App in ChatGPT

Business problem	Engineers ask the same questions about internal services — how to authenticate, which endpoint to use, who owns what — in chat channels, and the answers depend on who happens to be online. The documentation exists across wikis, repositories and service catalogues, and none of it is answerable in one place.

What to build	Build a ChatGPT app with the Apps SDK, backed by an MCP server that indexes a synthetic internal service catalogue, API references and design notes, and answers with citations to the specific document and version. Enforce access on the server so engineers only retrieve what their role allows. Ground library and API questions in current documentation, and refuse when the answer is not covered. Evaluate on a verified question set and measure specifically where the app is confidently wrong.
Success metrics	Answer accuracy on the verified set, citation correctness, confidently-wrong rate, access-control test pass, refusal correctness on uncovered questions.
Key techniques	• Apps SDK app over an MCP knowledge server • Server-side access control by role • Document- and version-level citation • Explicit measurement of confident errors • Verified question-set evaluation

30 TECHNOLOGY · AI/ML SOLUTIONS & APPLICATIONS

Support Ticket Classifier: Prompted Versus Distilled

Business problem	A support platform classifies tickets with a large model on every request. It works, the bill is growing faster than ticket volume, and nobody has tested whether a smaller model, a cached prompt or a fine-tuned model would hold the same accuracy at a fraction of the cost.
What to build	Build a ticket classification service with three implementations behind one API: a large model with a cached prompt, a smaller model with the same prompt, and a smaller model fine-tuned or distilled on labelled synthetic tickets. Evaluate all three on the same held-out set for accuracy, latency and cost per thousand tickets. Choose one with a stated decision rule and defend it. Ship the chosen version with an evaluation gate in CI, a canary against the current version and a tested rollback.
Success metrics	Accuracy, latency and cost per thousand tickets for each implementation, decision-rule defensibility, canary result, rollback success, blocked-merge demonstration.
Key techniques	• Prompt caching on a large-model baseline • Smaller-model and fine-tuned alternatives • Like-for-like evaluation on accuracy, latency and cost • Decision rule stated before results • Canary release with tested rollback

31 TECHNOLOGY · DATA ENGINEERING & DATA MODERNIZATION

Legacy ETL Repository Takeover with Codex

Business problem	A set of ETL jobs feeding product analytics has been patched by several teams over years. Jobs fail silently, nobody knows which outputs are still consumed, and every new metric is added as another job beside the old ones because changing an existing one is considered too risky.
What to build	Take a supplied ETL repository you did not write. Use Codex to map it, trace dependencies, find duplicated logic and drift from the intended structure, and reconstruct what each job guarantees — validating every reconstructed rule against tests and the supplied key. Write characterisation tests first, then consolidate duplicated transformations into shared components inside a controlled blast radius. Add data-quality checks as gates and prove output equivalence after every change, with Codex review and Codex Security on each pull request and a named human sign-off.

Success metrics	Rule-reconstruction accuracy against the key, duplicated logic removed, output equivalence after change, characterisation-test coverage, review defects caught before merge.
Key techniques	• Codex comprehension and drift detection on an unfamiliar repository • Characterisation tests before refactoring • Duplicate consolidation inside a blast radius • Data-quality checks as gates • Codex review and Codex Security with human sign-off

32 TECHNOLOGY · ANALYTICS & DECISION INTELLIGENCE

Natural-Language Metrics Service

Business problem	Product and business teams wait days for analysts to answer simple metric questions. Self-serve dashboards cover the common cases, and every other question becomes a ticket — or a spreadsheet built from an export with a definition nobody checked.
What to build	Build a service that answers metric questions over a synthetic product analytics warehouse by mapping the question to a governed semantic layer through function calling, generating SQL only against approved metric definitions, and executing it read-only. Validate generated queries before execution and refuse questions the semantic layer cannot answer. Return the number with its definition and the query behind it. Evaluate on a question set with verified answers, including ambiguous and unanswerable ones.
Success metrics	Answer exactness on verified questions, refusal correctness, query-validation catch rate on planted bad queries, p95 latency, definition-attribution coverage.
Key techniques	• Function calling to a governed semantic layer • Read-only SQL with validation before execution • Refusal on undefined metrics • Answer returned with definition and query • Verified question-set evaluation

33 TECHNOLOGY · PRICING & REVENUE OPTIMIZATION

AI Feature Cost-to-Serve and Pricing Simulator

Business problem	A software company has shipped AI features across its product and prices them as a flat add-on. Nobody knows what each customer actually costs to serve in model calls, so the heaviest users may be unprofitable and the pricing debate runs on averages.
What to build	Build metering that attributes model usage and cost per customer and per feature from synthetic tracing and usage data, reports unattributed cost as its own number, and reconciles attributed plus unattributed to the total. Build a simulator that tests pricing plans — tiers, usage caps, overage — against the actual customer base rather than a hypothetical one. Make the attribution method inspectable in the interface. Put the reconciliation invariant in CI and show it blocking a merge when a change breaks it.
Success metrics	Reconciliation exactness against total cost, unattributed share reported, simulation reproducibility, scenario computation time, blocked-merge demonstration.

Key techniques	• Usage and cost attribution from traces • Honest unattributed reporting • Pricing simulation against the actual base • Inspectable attribution method • Invariant testing as a merge gate

34 TECHNOLOGY · GENERATIVE AI & AGENTIC AI

AI-Powered Data Agent

Business problem	Business users depend on analysts to answer questions about their own data — what a metric means, why a number moved, how this quarter compares with the last. Every question becomes a request, answers arrive days later, and two analysts can report the same metric two ways. The data exists; the knowledge of what it means sits with a few people.
What to build	Propose a practical use case for a data agent inside a product or platform that business users already work in, and agree it with the panel before the BRD is written. Build an agent that connects to the data sources, using synthetic data in the real schema, analyses tables and relationships, and builds a knowledge layer of the available data, key metrics and business definitions that a data owner reviews and approves. Let business users ask questions in natural language and receive answers built on those approved definitions, through read-only queries validated before they run, returning each number with its definition and the query behind it, and declining when the data cannot answer. Generate charts, trend analysis and automated reports from the same definitions, so a figure in a report matches the figure in a conversation. Put a verified question set in CI as a merge gate and deploy with a demonstrated rollback.
Success metrics	Answer exactness on a verified question set, metric-definition coverage approved by the data owner, refusal correctness on unanswerable questions, chart and report accuracy against source, query-validation catch rate on planted bad queries, p95 response time, blocked-merge demonstration.
Key techniques	• Data-source analysis and an approved data knowledge layer • Governed metric and business definitions • Natural-language questions answered through read-only, validated queries • Chart, trend and report generation from the same definitions • Verified question-set evaluation as a CI merge gate

03 MANUFACTURING

7 capstone briefs. Briefs for plant and supplier systems, where the interface runs on the floor, the workflow touches systems that fail independently, and safety limits are not advisory. Each is scoped for a team of three to four from Day 3 to the panel, on synthetic data, and states the business problem, what to build, and the success metrics it will be judged on.

35 MANUFACTURING · SUPPLY CHAIN & OPERATIONS ANALYTICS

Supplier Delivery Risk Agent

Business problem	Purchasing teams learn about supplier disruptions — port closures, plant fires, insolvency — after a delivery is missed. The news is public days earlier, but nobody connects it to which open orders and which production lines depend on that supplier.

What to build	Build an agent that combines synthetic purchase order and bill-of-materials data, exposed through MCP tools, with web search over supplier and logistics news, and produces a ranked list of at-risk orders with the affected production lines and the evidence behind each. Cite every external claim, mark unverifiable ones, and never treat a single unconfirmed report as fact. Set run limits and a budget ceiling, and guard against instructions planted in web content. Evaluate on scenarios with planted disruptions and measure lead time against a missed-delivery baseline.
Success metrics	Planted-disruption detection rate and lead time, order-to-line impact accuracy, external citation correctness, injection-test pass, cost per run.
Key techniques	• Agent combining MCP tools and web search • Impact tracing from supplier to production line • Citation and verification of external claims • Injection defence on web content • Planted-scenario evaluation

36 MANUFACTURING · AI/ML SOLUTIONS & APPLICATIONS

Visual Defect Triage Service

Business problem	Inspection stations flag possible defects that a quality engineer reviews by image. Most flags are cosmetic, the queue backs up during peak shifts, and the real defects wait behind the false alarms.
What to build	Build a triage service that classifies synthetic defect images against a defect catalogue using a vision-capable model under a strict schema, returning the defect type, severity and the image region that supports it. Route low-confidence and high-severity cases to an engineer and clear only high-confidence cosmetic ones automatically. Serve it behind an API with health probes, and measure latency at shift-peak load. Evaluate against a labelled image set and a simple baseline, and gate merges on critical-defect recall.
Success metrics	Critical-defect recall with a target of zero misses, false-alarm reduction against baseline, severity accuracy, p95 latency at peak load, blocked-merge demonstration.
Key techniques	• Vision classification against a defect catalogue • Confidence and severity routing to engineers • Serving with health probes and load testing • Baseline comparison on a labelled image set • Recall gate on critical defects in CI

37 MANUFACTURING · GENERATIVE AI & AGENTIC AI

Maintenance Technician Voice Assistant

Business problem	A technician working on a machine needs one step of a long procedure while both hands are busy and the manual is on a terminal across the floor. Reading the wrong step for the wrong asset variant is a safety issue, not an inconvenience.
What to build	Build a hands-free voice assistant on the Realtime API that retrieves procedure steps from a synthetic manual corpus through a tool, after confirming the asset and variant with the technician. Preserve tables and conditional steps through parsing so torque values and conditions are not lost. Read one step at a time and wait for the technician to confirm before moving on. Refuse when the variant is not covered rather than reading the nearest procedure. Evaluate on near-miss variants and report precision at the top result.

Success metrics	Top-result precision on near-miss variants, variant-refusal correctness, step citation to procedure and revision, end-of-speech to response latency, noise-condition accuracy.
Key techniques	• Realtime API voice assistant with retrieval tool • Structure-preserving parsing of procedures • Variant confirmation before retrieval • Variant-aware refusal • Near-miss precision evaluation

38 MANUFACTURING · DATA ENGINEERING & DATA MODERNIZATION

Engineering Drawing and Bill-of-Materials Extraction

Business problem	Parts data for older products exists only in scanned drawings and bill-of-materials PDFs. Engineering changes, costing and spare-parts planning all depend on it, and every team that needs it re-keys a slightly different version.
What to build	Build a pipeline that extracts part numbers, quantities, materials and revision details from synthetic drawings and bill-of-materials PDFs with a vision-capable model under a strict schema, and loads them into a parts master. Validate part numbers against format rules, check quantities and hierarchy consistency, and stop a batch that fails rather than loading it. Track lineage from every record to its page and revision. Generate the per-format mapping code with Codex under AGENTS.md standards, with every mapping documented, typed and tested before merge.
Success metrics	Field-extraction accuracy on a labelled set, hierarchy-consistency check catch rate, lineage completeness, mapping test coverage, batch throughput and cost.
Key techniques	• Vision extraction from drawings and PDFs • Format and hierarchy checks as landing gates • Lineage to page and revision • Codex-generated mappings under enforced standards • Batch processing with cost tracking

39 MANUFACTURING · AI-POWERED AUTOMATION & WORKFLOWS

Legacy Plant System Automation with Computer Use

Business problem	A plant still records quality holds and releases in a legacy system with no API. Supervisors re-key the same information from the quality system into it every shift, and the two systems disagree often enough to cause audit findings.
What to build	Build a workflow that reads quality holds and releases from a mock quality system through an MCP tool and enters them into a mock legacy screen application using computer use, inside a sandboxed environment. Require approval before each submission, verify the entry by reading it back from the screen, and stop on any unexpected screen state rather than improvising. Log every action with a screenshot. Run it against screen changes, slow responses and pop-ups, and report which failures it detected and which it did not.
Success metrics	Entry accuracy verified by read-back, unexpected-state stop rate, approval-enforcement test pass, action-log completeness, re-keying time saved against baseline.

Key techniques	• Computer use for a system without an API • Sandboxed execution environment • Approval and read-back verification • Stop on unexpected screen state • Robustness testing against screen changes

40 MANUFACTURING · ANALYTICS & DECISION INTELLIGENCE

Shift Performance Analyst Agent

Business problem	Shift handovers and morning meetings rely on a summary someone writes from line logs, downtime records and quality data. KPIs are computed differently by different supervisors, and the narrative explains the shift in whichever way the writer saw it.
What to build	Build an agent that computes shift KPIs — output, availability, quality rate, downtime by cause — from synthetic line data with a documented method in code, and writes a handover summary in which every figure links to the calculation and records behind it. Flag anomalies against a stated baseline, and say when data for a line is missing rather than estimating. Evaluate KPI exactness against hand-calculated values and summary faithfulness with a calibrated judge.
Success metrics	KPI exactness against hand-calculated values, summary faithfulness, missing-data declaration correctness, anomaly-flag precision, preparation time against a manual baseline.
Key techniques	• KPI method documented and computed in code • Summaries with figure-to-record links • Anomaly detection against a baseline • Explicit missing-data declaration • Faithfulness evaluation with a calibrated judge

41 MANUFACTURING · AI & DATA STRATEGY / CONSULTING

Build-Versus-Configure Assessment for a Plant Assistant

Business problem	Plant leadership wants an AI assistant for operators and engineers, and the options range from configuring ChatGPT for the workforce, to building a custom application on the API, to not using AI for this at all. The expensive mistake is choosing on enthusiasm and discovering the cost, risk or adoption problem after the build.
What to build	Take a plant assistant request through the full analysis: user groups, tasks, data sensitivity and integration needs, then assess configured ChatGPT, a custom API application and a no-AI alternative against defined criteria — capability, cost at real volume, risk, governance and time to value. State plainly which option fits which task, and where the answer is not to use AI. Build the cost, risk and value case with sensitivity on the assumptions that move it. Then build a working proof of the recommended option for one task, to the same engineering, security and deployment standard as every other project.
Success metrics	Requirement completeness, defensibility of the recommendation under panel challenge, cost bounds with sensitivity ranges, working proof delivered for the selected task.

Key techniques	• Requirements and task analysis by user group • Option assessment against defined criteria • Cost, risk and value case with sensitivity analysis • Do-not-use-AI conclusions where justified • Working proof of the recommended option